School Management System — Technical Specification

Owner: Reggycodas **Version:** 1.0 (October 2026) **Audience:** Development team / Replit Agent

> **Instructions for the developer/agent:** Build this system in the phases listed in Section 14. Complete and test each phase before starting the next. Do not add modules listed as out of scope. Where this document says "configurable", store the value in the database per school, not in code.

---

1. Product overview

A lean, multi-school (multi-tenant) web system for Kenyan schools. The core purpose is **fee collection and automatic reconciliation**: every payment received through M-Pesa or a bank must be posted to the correct student's account automatically, with a clear exception queue for anything that cannot be matched with confidence.

Secondary modules: CBC/CBE assessments and report cards, attendance, and a REST API consumed by the **LeadsIntel WhatsApp bot** so parents can check balances, pay, and receive receipts and results.

1.1 In scope

1. Schools, users and roles
2. Students, classes, streams, parents, sibling linking
3. Fee structures, invoicing, student ledgers
4. Payments: M-Pesa (C2B Paybill + STK Push), bank APIs, cash, cheque
5. Reconciliation engine (matching, exception queue, daily statement check)
6. Payment plans (installments) and reminders
7. Bot API for LeadsIntel WhatsApp bot
8. CBC/CBE assessments, report cards
9. Attendance
10. Approvals and audit log
11. Reports and exports

1.2 Out of scope (do NOT build)

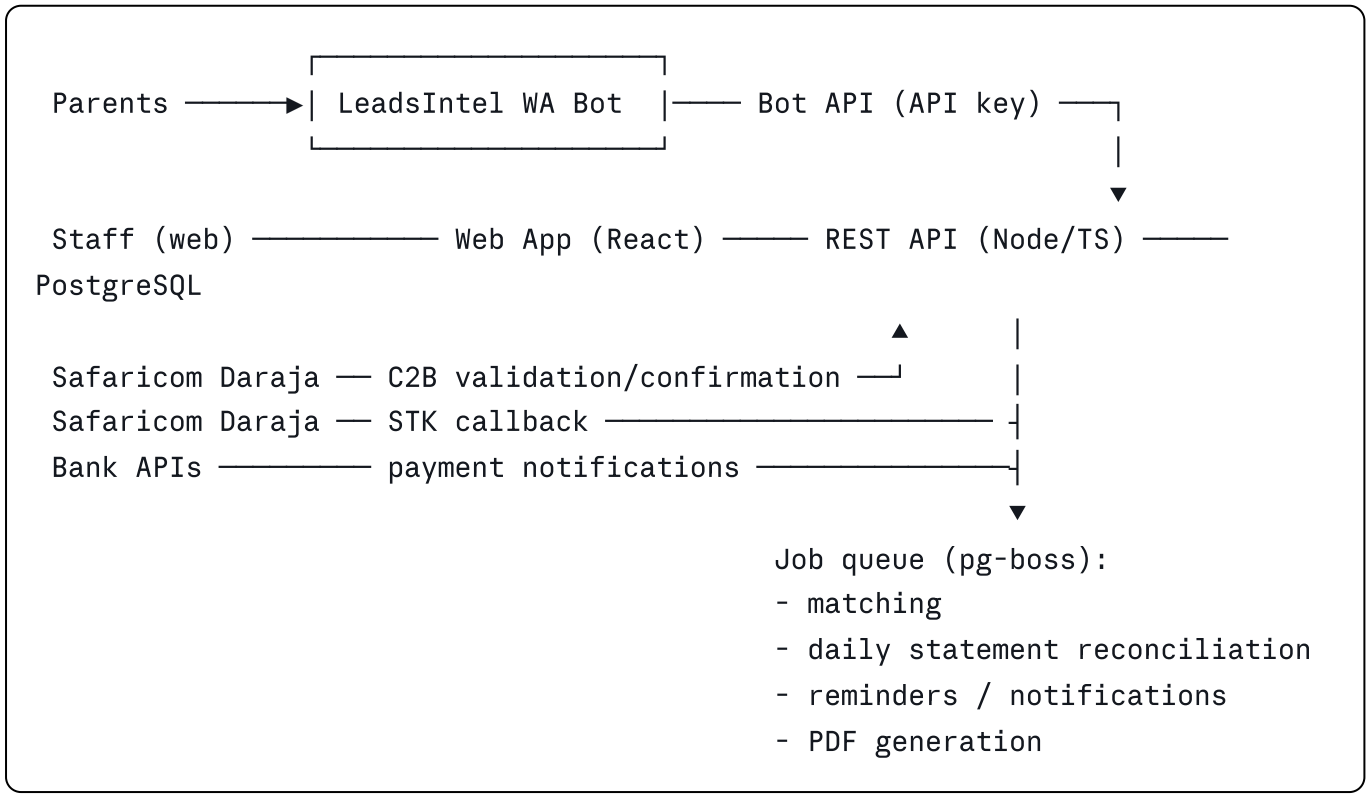
Payroll, inventory/stores, library, hostel management, transport fleet management, timetabling, lesson plans, schemes of work, full portfolio management, general accounting/general ledger, procurement, HR.

---

2. Tech stack

Layer	Choice	Notes
Language	TypeScript (Node.js 20+)	Same language front and back
Backend	Express or Fastify	REST JSON API
Database	PostgreSQL	Use Replit's managed Postgres
ORM	Prisma	Migrations in repo
Background jobs	pg-boss (Postgres-backed queue)	No Redis needed
Frontend	React + Vite + Tailwind CSS	Responsive; teacher screens must work on phones
Auth	Email/phone + password, JWT (httpOnly cookie)	bcrypt/argon2 hashing
PDF generation	pdfmake or @react-pdf/renderer	Receipts, statements, report cards
Excel import/export	exceljs	Student import, report export
Validation	zod	All request bodies
File storage	Replit Object Storage (or S3-compatible)	Evidence photos, generated PDFs
Timezone	Africa/Nairobi	Store timestamps in UTC, display in EAT
Currency	KES	Store money as `DECIMAL(14,2)`; never use floats

3. Architecture



3.1 Multi-tenancy

- Every tenant-owned table has `school_id`. Every query must be scoped by `school_id` from the authenticated user (never from the request body).
- Each school has its own M-Pesa shortcode and bank account configuration.
- Incoming M-Pesa callbacks are routed to a school by `BusinessShortCode`; bank callbacks by account number.

3.2 Backend module structure

```
src/
  modules/
    auth/ schools/ users/ students/ parents/ classes/
    fees/            # fee structures, items, invoices
    ledger/          # ledger entries, balances
    payments/        # payment records, allocation
    reconciliation/  # matching engine, exception queue, daily recon
    integrations/
      mpesa/         # Daraja C2B, STK, pull transactions
      banks/         # adapter interface + per-bank adapters
    payment-plans/
    bot-api/         # endpoints for LeadsIntel
    notifications/   # WhatsApp (via LeadsIntel), SMS fallback
    academics/       # CBC curriculum, assessments, report cards
    attendance/
    approvals/
    audit/
    reports/
  jobs/
  lib/ (money, phone normalisation, pdf, excel)
```

---

4. Roles and permissions

Permission	Super Admin (platform)	School Admin	Principal	Bursar	Teacher	Class Teacher
Manage schools / subscriptions	✓					
School settings, users		✓				
Students, classes, parents		✓	view	✓	view own classes	view own class
Fee structures, invoicing		✓	view	✓		
Record cash/cheque payments				✓		
Assign unmatched payments				✓		
Request waiver/discount/refund				✓		
Approve waiver/discount/refund		✓	✓			
Payment plans			✓ approve	✓ create		
Enter assessments					✓ own subjects	✓
Report card comments			✓ head remarks			✓
Attendance			view			✓
Finance reports		✓	✓	✓		
Academic reports		✓	✓		own classes	own class
Audit log	✓	✓	✓			

Rule: **a user cannot approve their own request** (maker-checker).

5. Data model

All tables include `id (uuid)`, `created_at`, `updated_at`. Tenant tables include `school_id`. Soft-delete with `deleted_at` where noted; **never** delete financial records.

5.1 Core

schools: name, code, county, type (`primary|junior|senior|comprehensive|other`), logo_url, address, phone, email, mpesa_shortcode, mpesa_config (encrypted json), settings (json: matching thresholds, sibling split rule, reminder schedule, term dates), status.

users: school_id (nullable for super admin), name, email, phone, password_hash, role, is_active, last_login_at.

academic_years: school_id, name (e.g. 2027), start_date, end_date, is_current.

terms: school_id, academic_year_id, name (Term 1/2/3), start_date, end_date, is_current.

grades (levels): school_id, name (e.g. Grade 7, Form 3), level_band (`pre_primary|lower_primary|upper_primary|junior|senior|other`), order.

streams: school_id, grade_id, name (e.g. East), class_teacher_user_id.

5.2 Students and parents

students: school_id, admission_no (unique per school), first_name, middle_name, last_name, gender, date_of_birth, grade_id, stream_id, status (`active|transferred|graduated|left`), boarding_status (`day|boarder`), transport_route_id (nullable), pathway (nullable, senior school), admission_date, upi/assessment_no (nullable), photo_url.

guardians: school_id, name, phone (normalised `2547XXXXXXXX` / `2541XXXXXXXX`), alt_phone, email, relationship, id_number (optional — store only if school needs it).

student_guardians: student_id, guardian_id, is_primary, receives_notifications.

families: school_id, name (e.g. "Kamau family"). Siblings share a `family_id` on students. Created automatically when one guardian is linked to more than one student.

payer_aliases (learned matching rules): school_id, alias_type (`phone|reference|name|bank_account`), alias_value (normalised), student_id (nullable), family_id (nullable), source (`bursar_assigned|parent_confirmed|guardian_record`), confidence, created_by.

5.3 Fees and billing

fee_items: school_id, name (Tuition, Lunch, Activity, Transport, Boarding…), code, priority (allocation order, 1 = paid first), is_optional.

fee_structures: school_id, academic_year_id, term_id, grade_id, boarding_status (nullable = applies to all), name, status (`draft|approved|archived`), approved_by, approved_at,

version.

fee_structure_lines: fee_structure_id, fee_item_id, amount.

student_optional_charges: student_id, fee_item_id, amount, term_id (nullable = every term), start_date, end_date.

invoices: school_id, invoice_no (sequential per school, e.g. INV-2027-000123), student_id, term_id, issue_date, due_date, total_amount, status (`open|partially_paid|paid|cancelled`), type (`term|one_off`), notes.

invoice_lines: invoice_id, fee_item_id, description, amount.

adjustments (discounts, waivers, bursaries, write-offs, reversals): school_id, student_id, invoice_id (nullable), type (`discount|sibling_discount|waiver|bursary|sponsorship|write_off|correction`), amount, reason, reference (e.g. BOM minute, sponsor ref), sponsor_id (nullable), status (`pending|approved|rejected`), requested_by, approved_by, approved_at.

sponsors: school_id, name, type (`ngcdf|county|ngo|church|individual|company|other`), contact.

5.4 Ledger (source of truth for balances)

ledger_entries (append-only, never updated or deleted):

- school_id, student_id, entry_date, type (`invoice|payment|adjustment|refund|reversal|credit_transfer|opening_balance`)
- debit (increases what parent owes), credit (reduces what parent owes)
- reference_type + reference_id (invoice, payment, adjustment…)
- term_id, description, created_by

Student balance = SUM(debit) − SUM(credit). Negative balance = credit (overpayment). Maintain a cached `student_balances` table (student_id, balance, updated_at) refreshed in the same transaction as each ledger write. Provide a nightly job that recomputes from ledger and alerts on any mismatch.

Corrections are made with reversal entries, never by editing an existing entry.

5.5 Payments

payment_transactions (raw money received, one row per provider transaction):

- school_id, channel (`mpesa_c2b|mpesa_stk|bank|cash|cheque`)
- provider (`safaricom|equity|kcb|coop|…|manual`), provider_txn_id (M-Pesa receipt e.g. `SJK3XXXXXX`, bank ref)
- **UNIQUE (school_id, provider, provider_txn_id)** — idempotency

- amount, transaction_time, payer_phone (normalised), payer_name, account_reference (raw), account_reference_normalised, bank_account_from, narrative
- raw_payload (jsonb)
- status (`received|matched|partially_allocated|unmatched|suspended|refunded|reversed`)
- match_method, match_confidence, matched_by (`auto|bursar|parent_bot`), matched_at
- source (`callback|statement_recon|manual`)
- receipt_no (sequential per school)

payment_allocations: payment_transaction_id, student_id, amount, allocated_by, allocated_at. (One payment can be split across siblings — sum of allocations ⩽ payment amount.)

unmatched_queue: payment_transaction_id, candidates (jsonb: list of {student_id, score, reasons}), reason (`no_reference|invalid_reference|multiple_siblings|low_confidence|amount_exceeds`), bot_prompt_sent_at, resolved_at, resolved_by, resolution (`assigned|split|refund|suspense`).

cheques: payment_transaction_id, cheque_no, bank, drawer, status (`received|banked|cleared|bounced`). Bounced cheque → automatic reversal entry.

refunds: school_id, student_id, payment_transaction_id (nullable), amount, method, status (`pending|approved|paid|rejected`), requested_by, approved_by.

5.6 Payment plans

payment_plans: school_id, student_id, term_id, total_amount, status (`proposed|active|completed|defaulted|cancelled`), requested_via (`bot|office`), approved_by. **payment_plan_installments**: plan_id, due_date, amount, amount_paid, status (`upcoming|due|paid|partially_paid|overdue`).

5.7 Communication

notifications: school_id, guardian_id, channel (`whatsapp|sms|email`), template, payload, status (`queued|sent|delivered|failed`), provider_message_id, sent_at, error. **otp_codes**: phone, code_hash, purpose, expires_at, attempts.

5.8 Academics (CBC/CBE)

learning_areas: school_id, grade_id (or level_band), name, code, is_core, pathway (nullable). **strands**: learning_area_id, name, order. **sub_strands**: strand_id, name, order. **performance_scales**: school_id, name (e.g. "4-Level Rubric", "8-Level Junior Scale"), is_default. **performance_levels**: scale_id, code (EE, ME, AE, BE / EE1, EE2, ME1, ME2, AE1,

AE2, BE1, BE2), label, points, min_percent, max_percent, order. **grade_scale_assignments**: grade_id, scale_id. (Which scale each grade uses — configurable, because KNEC adjusts scales.) **teacher_assignments**: user_id, stream_id, learning_area_id, term_id. **assessments**: school_id, term_id, grade_id, learning_area_id (nullable for multi-subject exams), name (CAT 1, Mid-term, End-term, Project), type (`formative|summative|project|sba`), max_marks (nullable for rubric-only), weight, date, status (`open|locked`). **assessment_results**: assessment_id, student_id, sub_strand_id (nullable), marks (nullable), performance_level_id, comment, entered_by, entered_at. **evidence**: assessment_result_id, file_url, caption. **competency_ratings**: student_id, term_id, competency (`communication|critical_thinking|creativity|citizenship|digital_literacy|learning_to_learn|self_efficacy`), performance_level_id. **value_ratings**: student_id, term_id, value (`love|responsibility|respect|unity|peace|patriotism|social_justice|integrity`), performance_level_id. **report_cards**: student_id, term_id, class_teacher_comment, head_teacher_comment, file_url, status (`draft|published|sent`), published_at.

5.9 Attendance

attendance_records: school_id, student_id, stream_id, date, status (`present|absent|late|excused`), marked_by. UNIQUE(student_id, date).

5.10 Control

approvals: school_id, entity_type, entity_id, action, requested_by, decided_by, decision, reason, decided_at. **audit_logs** (append-only): school_id, user_id (or `system`/`bot`), action, entity_type, entity_id, before (jsonb), after (jsonb), ip, user_agent, created_at. **integration_logs**: school_id, provider, direction (`in|out`), endpoint, request, response, status_code, created_at (redact secrets).

---

6. Business rules

6.1 Invoicing

- Bursar generates term invoices per grade (or whole school) from the **approved** fee structure. Draft structures cannot be billed.
- Each student's invoice = fee structure lines matching grade + boarding status + their optional charges.
- Generation is idempotent: running it twice for the same student and term must not create duplicates.
- Approved recurring discounts (e.g. sibling discount) apply automatically as adjustment entries when invoices are generated.
- One-off charges create a separate invoice of type `one_off`.
- Previous balance is **not** copied into the new invoice — it remains in the ledger, and statements show balance brought forward.

6.2 Payment allocation (within one student)

1. Apply to the oldest open invoice first.
2. Within an invoice, allocate across fee items by `fee_items.priority`.
3. Any excess becomes credit (negative balance), automatically consumed by the next invoice.
4. Store per-fee-item allocation so votehead/fee-item collection reports are accurate.

6.3 Approval rules

- Waivers, discounts, write-offs, refunds, corrections and payment reversals require approval (Principal or School Admin) and cannot be approved by the requester.
- Bursaries/sponsorships are recorded as `bursary`/`sponsorship` adjustments linked to a sponsor — **never** as discounts.
- Fee structure changes after approval create a new version; existing invoices are not changed retroactively.

6.4 Receipts

- Every posted payment gets a sequential receipt number per school and a PDF receipt.
- No manual receipt numbers. Cash and cheque payments can only be receipted through the system.
- Receipt is sent to the primary guardian via WhatsApp (LeadsIntel) immediately after posting.

6.5 Payment plans

- Created by bursar or requested through the bot; activated after Principal approval (or auto-approved if the school setting allows plans under a set amount).
- Payments allocate normally; a job updates installment statuses daily.
- Reminders: 3 days before, on the due date, and 3 days after (configurable).
- Plan status `defaulted` after N missed installments (configurable).

---

7. Reconciliation engine (core of the system)

7.1 Pipeline

```
receive → store raw (idempotent) → normalise → match → (auto-post | queue) →
notify → learn
```

Every incoming payment from any source (C2B callback, STK callback, bank notification, statement recon, manual) goes through the **same pipeline** as a pg-boss job.

7.2 Normalisation

- **Reference:** uppercase; trim; remove spaces, `-`, `/`, `.`, `#`; strip prefixes `ADM`, `ADMNO`, `ADMISSION`, `NO`, `REG`; compare both with and without leading zeros.

- **Phone:** convert `07XXXXXXXX`, `+2547…`, `7XXXXXXXX` → `2547XXXXXXXX` (same for `01…` → `2541…`). Note: M-Pesa C2B may return masked or hashed MSISDNs; store what is received and match on it only if it is a full number.
- **Name:** uppercase, remove punctuation, split into tokens.

7.3 Matching rules (evaluated in order, scores configurable per school)

#	Rule	Score
1	Reference = exact admission number of an active student	100
2	Normalised reference = normalised admission number	95
3	Reference or phone matches a `payer_alias` linked to one student	90
4	Payer phone matches a guardian with exactly one active student	85
5	Payer phone matches a guardian with multiple students (siblings)	→ sibling flow (7.5)
6	Payer name fuzzy-matches guardian name (Jaro-Winkler ⩾ 0.90) and student surname	70
7	Reference fuzzy-matches an admission number (edit distance 1, e.g. typo)	65
Bonus	Amount equals the candidate's outstanding balance or next installment	+10
Penalty	Candidate student is not `active`	−30

- **Auto-post threshold:** score ⩾ 85 (configurable). Post, receipt, notify.
- Below threshold: create `unmatched_queue` row with top 5 candidates and reasons.
- If two candidates tie above threshold → send to queue (never guess).

7.4 Exception queue (bursar screen)

- List of unmatched payments with candidates, scores and reasons.
- Actions: **Assign** to one student, **Split** across students, **Mark as suspense** (unknown, keep for follow-up), **Refund** (requires approval).
- On assign: post allocation, send receipt, and create a `payer_alias` (phone → student/family, reference → student) so future payments match automatically.
- Aging: highlight items older than 3 days.

7.5 Sibling / family payments

When a payment maps to a family with several active students and no clear student reference:

1. If school setting `sibling_split_rule = ask_parent` (default): bot sends the guardian a message listing the children and balances and asks how to split (options: one child, or "split by balances"). Response posts the allocation.
2. If no reply in 24 hours (configurable) → apply the fallback rule:
   - `proportional_to_balance` (default fallback), or
   - `oldest_arrears_first`, or
   - leave in queue for bursar.
3. A reference that matches one sibling's admission number always goes to that sibling (rule 1/2 wins).

7.6 Lump-sum sponsor / bursary payments

- Bursar selects the payment, chooses a sponsor, uploads an Excel/CSV (admission_no, amount) or picks students on screen.
- System validates: sum of lines = payment amount (or remainder held as unallocated with reason), all admission numbers exist.
- Posts `bursary`/`sponsorship` credit entries per student and generates an **acknowledgement letter PDF** for the sponsor listing students and amounts.

7.7 Daily statement reconciliation (job, 02:00 EAT)

1. For each school and each connected channel, pull the previous day's transactions (M-Pesa Pull Transactions API; bank statement/transactions API).
2. Compare against `payment_transactions` for that day by provider_txn_id.
3. **In statement but not in system** (missed callback) → ingest through the normal pipeline with `source = statement_recon`.
4. **In system but not in statement** → flag for review (possible fake/duplicate entry).
5. **Amount mismatch** → flag.
6. Produce a reconciliation summary (counts, totals by channel, discrepancies) visible on the bursar dashboard and in reports.

7.8 Duplicates and reversals

- Unique constraint on `(school_id, provider, provider_txn_id)`; duplicate callbacks return success but do nothing.
- Possible duplicate payments by parents (same phone, same amount, within 10 minutes) → post both but flag on dashboard.
- M-Pesa reversals and bounced cheques → reversal ledger entries linked to the original payment; parent notified.

---

8. Integrations

> Confirm all endpoint paths, payloads and field names against the current Safaricom Daraja and bank developer documentation before implementation. Build each

> integration behind an adapter interface so providers can be added or changed without touching business logic.

8.1 M-Pesa (Safaricom Daraja)

Per-school config (encrypted): consumer_key, consumer_secret, shortcode, passkey (STK), environment (`sandbox|production`), initiator credentials if needed.

a) C2B Paybill

- Register Validation and Confirmation URLs per shortcode:
  - `POST /api/integrations/mpesa/c2b/validation/:schoolToken`
  - `POST /api/integrations/mpesa/c2b/confirmation/:schoolToken`
  - `:schoolToken` = random unguessable per-school token.
- **Validation:** look up the account reference. If no matching student (after normalisation, and no matching alias), reject with Daraja's invalid-account result code (C2B00012) so the parent sees the error before paying. Respond within Daraja's timeout window; if the lookup fails internally, **accept** (never block a parent because of our own error). External validation must be enabled on the shortcode by Safaricom.
- **Confirmation:** respond success immediately, then queue the matching job.

b) STK Push (Lipa na M-Pesa Online) — used by the bot and the web parent statement link

- `POST /api/payments/stk` (internal) → calls Daraja STK push with `AccountReference = admission_no` (or family code for split payments).
- Callback: `POST /api/integrations/mpesa/stk/callback/:schoolToken`.
- Store `CheckoutRequestID`; if no callback within 2 minutes, call STK Query API.
- STK payments made for a known student skip matching (score 100).

c) Pull Transactions / Transaction Status

- Used by the daily reconciliation job and to verify suspicious callbacks.

Security: callbacks are not signed — protect with per-school path tokens, optional Safaricom IP allowlist, idempotency, and verification of high-value or unusual transactions with the Transaction Status API.

8.2 Banks

Implement a common interface:

```ts
```

```ts
interface BankAdapter {
  name: string;
  verifyWebhook(req): boolean;                 // signature / secret check
  parseNotification(payload): NormalisedPayment;
  fetchTransactions(account, from, to): Promise<NormalisedPayment[]>;
}
```

- Webhook: `POST /api/integrations/banks/:provider/webhook/:schoolToken`.
- Start with one bank (whichever the pilot school uses), then add others. Candidate APIs: Equity (Jenga), KCB (Buni), Co-operative Bank — confirm current availability of collection notifications and statement APIs for each.
- Where the bank supports per-student virtual account numbers or payment references, map them to students via `payer_aliases` (`alias_type = bank_account`).
- **Fallback:** bank statement upload (CSV/Excel/MT940) with a column-mapping screen, processed through the same pipeline.

8.3 LeadsIntel WhatsApp bot

The school system exposes a **Bot API**; the LeadsIntel bot calls it. The school system sends outbound messages by calling the LeadsIntel send-message API.

Authentication: per-school API key in header `X-Api-Key` + HMAC signature of the body (`X-Signature`, SHA-256 with shared secret) + timestamp header to prevent replay.

Parent identity: the bot passes the WhatsApp phone number. If it matches a guardian, proceed. If not, return `unknown_guardian`. For sensitive actions (statements, results) on first use, verify with a 6-digit OTP sent to the phone on record (valid 10 minutes, max 5 attempts).

Bot API endpoints (`/api/bot/v1`):

Method	Path	Purpose
POST	`/identify`	phone → guardian + list of children (name, grade, admission no)
POST	`/otp/request`, `/otp/verify`	OTP flow
GET	`/students/:id/balance`	Current balance, last payment, next due
GET	`/students/:id/statement?term=`	Statement summary + PDF link (signed, expires 24h)
POST	`/payments/stk`	Start STK push {student_id or family split, amount, phone}
GET	`/payments/:id/status`	STK status
GET	`/unmatched/pending`	Payments from this phone awaiting assignment
POST	`/unmatched/:id/resolve`	Parent chooses child or split
POST	`/payment-plans/request`	Parent requests a plan {student_id, installments, dates}
GET	`/students/:id/results?term=`	Report card summary + PDF link
GET	`/students/:id/attendance?month=`	Attendance summary
GET	`/school/info`	Fee structure summary, paybill, term dates, contacts

Outbound messages (system → LeadsIntel → parent): payment receipt, unmatched-payment question, fee reminder, installment reminder, invoice issued, report card ready, absence alert, school announcement. Use templates stored per school; respect WhatsApp template rules for messages outside the 24-hour window. Fall back to SMS if WhatsApp delivery fails (SMS provider behind an adapter, e.g. Africa's Talking).

---

9. CBC / CBE module rules

- Curriculum (learning areas → strands → sub-strands) is configurable per school, with a seed dataset per level that schools can edit.
- Performance scales are data, not code: seed the 4-level rubric (EE, ME, AE, BE) and the 8-level junior school scale (EE1, EE2, ME1, ME2, AE1, AE2, BE1, BE2) with points and percentage bands; schools assign a scale per grade.

- **Formative entry:** class grid — rows = learners, columns = sub-strands (or one assessment); tap a level per cell; autosave; works offline-tolerant (queue saves and retry) on mobile.
- **Summative entry:** marks out of `max_marks`; level computed from the scale's percentage bands; teacher can override with a reason (logged).
- **Term result per learning area:** weighted average of assessments in the term (weights set per assessment); converted to a level.
- Assessments can be **locked** by the class teacher/admin; locked results need admin unlock (logged).
- **SBA/KNEC export:** export assessment results per grade/learning area to CSV/Excel in a configurable column layout so it can be uploaded to the KNEC portal. Keep the layout configurable because the portal format can change.
- **Report card (PDF):** school header/logo, learner details, learning area levels (and marks where applicable), competencies, values, attendance summary, class teacher and head teacher remarks, next term opening date. Fee balance is **not** printed on report cards and results are **never** withheld because of fees.
- **Analytics:** per class, count of learners at each level per learning area; per learner, trend across terms.

---

10. Attendance

- Class teacher marks the daily register (default all present; tap to change).
- Absent learners trigger an absence alert to the guardian (configurable: on/off, send time).
- Reports: daily summary, learner attendance %, chronic absentees list.

---

11. Screens (web app)

Bursar / Admin

1. Dashboard: today's collections by channel, term collection vs billed, unmatched count, recon status, defaulters count
2. Students list (search by name/admission no/phone), student profile (details, guardians, siblings, ledger, invoices, payments, plans, results)
3. Bulk import students (Excel template download → upload → validation preview → commit)
4. Promote students (end of year)
5. Fee items, fee structures (draft → approve), optional charges
6. Generate invoices (choose term/grades → preview → generate)
7. Payments list (filters: channel, date, status)
8. **Unmatched queue** (assign / split / suspense / refund)

9. Record cash / cheque payment
10. Sponsor lump-sum allocation
11. Adjustments and approvals inbox
12. Payment plans
13. Reconciliation report (daily recon results)
14. Reports (Section 12)
15. Settings: school profile, terms, M-Pesa and bank config, matching thresholds, sibling split rule, reminders, message templates, users

Teacher 16. My classes, assessment entry grid, report card comments, attendance register (all mobile-friendly)

Principal 17. Dashboard (finance + academics summary), approvals inbox, report card head remarks, reports

---

12. Reports (all exportable to Excel and PDF)

1. Collections by day / date range / term, by channel
2. Collections by fee item
3. Student balances by class (with balance brought forward)
4. Defaulters list (filters: minimum balance, class, with/without active plan)
5. Aged arrears (current, 1 term, 2 terms, 3+ terms)
6. Unmatched / suspense payments
7. M-Pesa and bank reconciliation report
8. Adjustments register (discounts, waivers, bursaries with references and approvers)
9. Sponsor/bursary report per sponsor
10. Payment plans status
11. Student fee statement (PDF, per student)
12. Class performance summary, learner progress
13. Attendance summary

---

13. Security, privacy and reliability

- HTTPS only; secrets in environment variables; integration credentials encrypted at rest (AES-256-GCM with key from env).
- Password hashing (argon2/bcrypt); login rate limiting; optional 2FA for admin/bursar/principal.
- Strict `school_id` scoping in every query (add automated tests that try cross-tenant access).
- Audit log for every create/update on financial, student and academic records, and every login.

- Ledger and payment tables are append-only at the application level; no hard deletes.
- Kenya Data Protection Act 2019: collect only needed data, role-based access to learner data, signed and expiring links for PDFs, ability to export/delete a guardian's personal data on request (financial records retained, personal data anonymised).
- Webhook endpoints: respond fast (< 2s), process asynchronously; log all raw payloads to `integration_logs` (with secrets redacted).
- Jobs retried with backoff; failed jobs visible on an admin screen.
- Daily automated database backup; documented restore procedure.
- Health check endpoint `/api/health`.

---

14. Build phases and acceptance criteria

Phase 1 — Foundation

Schools, users, roles, auth, academic years/terms, grades/streams, students, guardians, sibling/family linking, Excel import, audit log. **Done when:** an admin can create a school, import 500 students with guardians from Excel, and siblings are auto-linked by shared guardian phone.

Phase 2 — Fees and ledger

Fee items, fee structures (approval), optional charges, invoice generation, ledger, adjustments with approval, cash/cheque payments, receipts (PDF), student statement (PDF). **Done when:** invoices generate for a term without duplicates; a cash payment allocates by priority; balances equal ledger sums; nightly balance check passes.

Phase 3 — M-Pesa and reconciliation

Daraja C2B (validation + confirmation), STK push + query, matching engine, unmatched queue, payer alias learning, sibling split, sponsor lump-sum allocation, duplicate protection, daily Pull Transactions reconciliation. **Done when:** all test scenarios in Section 16 pass in Daraja sandbox.

Phase 4 — Bot API and notifications

Bot API, OTP, outbound messages via LeadsIntel, SMS fallback, reminders, payment plans. **Done when:** a parent can identify, check balance, pay by STK, receive a receipt, resolve an unmatched payment, and request a payment plan entirely on WhatsApp.

Phase 5 — Bank integration

Bank adapter interface, first bank adapter, statement upload fallback, bank reconciliation. **Done when:** bank notifications post to students through the same pipeline and the daily bank recon flags missing transactions.

Phase 6 — Academics and attendance

Curriculum setup and seed data, scales, assessments, entry grids, weighted results, report cards, KNEC export, attendance and absence alerts. **Done when:** a class teacher can enter a

term's results on a phone, publish report cards, and parents receive them on WhatsApp.

Phase 7 — Reports and polish

All reports, dashboards, approvals inbox, performance tuning, backup/restore test.

---

15. Environment variables

```
DATABASE_URL=
JWT_SECRET=
ENCRYPTION_KEY=                # 32-byte key for credential encryption
APP_BASE_URL=                  # used to build callback URLs
MPESA_ENV=sandbox|production
LEADSINTEL_API_URL=
LEADSINTEL_API_KEY=
LEADSINTEL_WEBHOOK_SECRET=
SMS_PROVIDER=africastalking
SMS_API_KEY=
SMS_USERNAME=
SMS_SENDER_ID=
OBJECT_STORAGE_BUCKET=
TZ=Africa/Nairobi
```

(Per-school M-Pesa and bank credentials are stored encrypted in the database, not in env.)

---

16. Reconciliation test scenarios (must pass)

1. Parent pays with exact admission number → auto-posted, receipt sent.
2. Parent types `adm 0123` for admission `123` → auto-posted (normalised).
3. Parent types a non-existent admission number at Paybill → rejected at validation.
4. Validation endpoint times out/errors → payment accepted, then handled by matching.
5. Blank reference, phone belongs to guardian with one child → auto-posted.
6. Blank reference, phone belongs to guardian with three children → bot asks parent; parent chooses split → three allocations posted.
7. Same as 6 but parent does not reply in 24h → fallback rule applied.
8. Reference is one digit off an admission number (typo) → queued with that student as top candidate.
9. Unknown phone and unknown reference → queued; bursar assigns; next payment from that phone auto-matches.
10. Same M-Pesa confirmation received twice → only one payment recorded.
11. Callback missed entirely → daily Pull Transactions job ingests it.
12. Payment larger than balance → excess held as credit and applied to the next invoice.

13. Sponsor pays KES 200,000 for 20 students via bank → bursar uploads list, 20 bursary entries posted, acknowledgement PDF generated.
14. Cheque bounces → reversal posted, balance restored, guardian notified.
15. Bursar requests a waiver and tries to approve it → blocked.
16. User from School A tries to read School B's payment → 403.

---

17. Seed / demo data

Provide a seed script with: one demo school, 3 grades × 2 streams, 60 students, 45 guardians (including families with 2–3 siblings), a Term 1 fee structure, generated invoices, sample M-Pesa transactions covering scenarios in Section 16, the 4-level and 8-level scales, and one learning area with strands and sub-strands per grade.